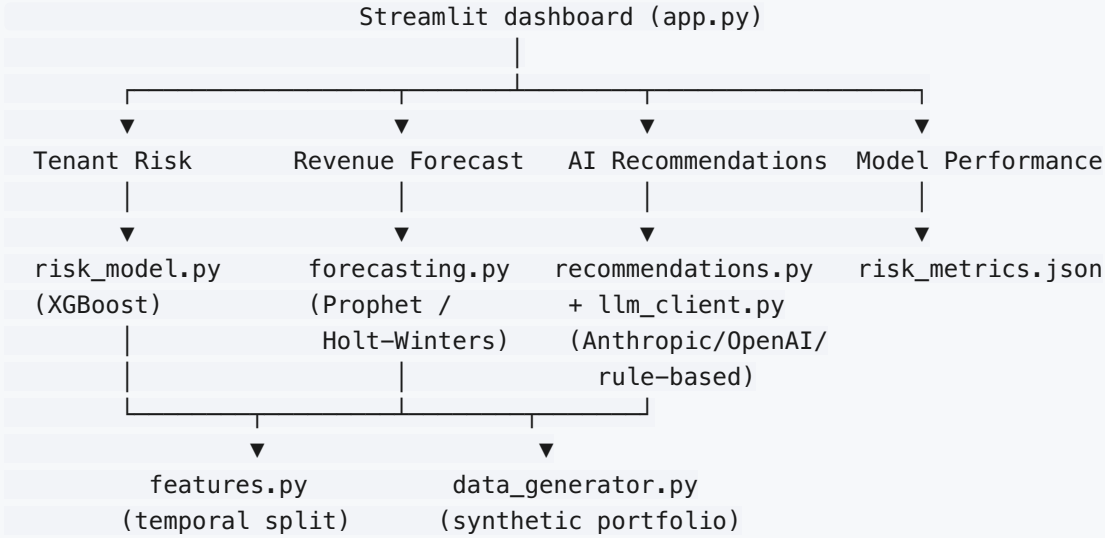
Technical Approach Report — TenantEase AI Intelligence Engine

Prepared for: Cynbit Technologies — AI/ML Engineer Intern assessment **Author:** Krishna Reddy Nalla **Date:** 19 October 2026 **Companion:** AI Opportunity Report · proof of concept in this repository

---

1. System architecture

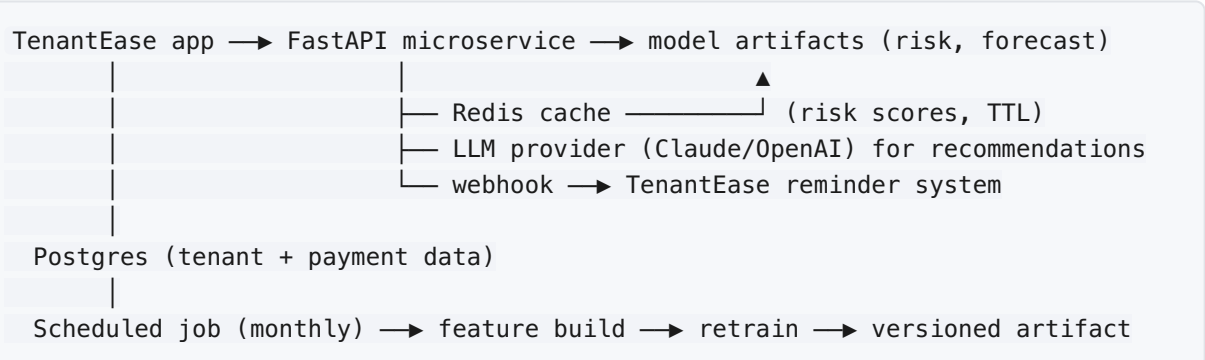
1.1 Proof-of-concept architecture



Each layer is an independent, importable module with a single responsibility. The dashboard is a thin presentation layer; all logic lives in `src/` and is unit-tested without Streamlit.

1.2 Production architecture (proposed)

In production the same three models would run as a service behind TenantEase:



- **FastAPI** serves `/score`, `/forecast`, `/recommend` endpoints.

- **Redis** caches risk scores (they change slowly) with a TTL keyed to the monthly retraining cadence.
- **Scheduled retraining** rebuilds features and retrains monthly as new payment history accrues; artifacts are versioned so a bad model can be rolled back.
- **Webhook** lets the risk layer trigger TenantEase's *existing* reminder system earlier for high-risk tenants — reusing infrastructure, not rebuilding.
- An **LLM + RAG** path (LangChain + a vector store such as Pinecone over lease documents) would let recommendations cite lease-specific clauses.

---

2. Model selection rationale

2.1 Layer 1 — XGBoost (classification)

Choice	Why
XGBoost over logistic regression	Payment behaviour has non-linear interactions (rent burden only bites once the delay trend is also rising). Trees capture this; it also yields gain-based feature importances for explainability.
Benchmarked against logistic regression	The XGBoost AUC is never quoted alone — we report the baseline's CV and test AUC too (0.864 test). XGBoost lands at 0.893.
Calibrated probabilities, no `scale_pos_weight`	The 0–100 risk score *is* the probability × 100, so calibration matters. Instead of reweighting, we tune the operating threshold.

2.2 Layer 2 — Prophet (time series)

Choice	Why
Prophet	Monthly series with real seasonality (festival-season squeeze, FY-end outflow); Prophet handles trend changepoints and emits confidence intervals natively — essential for cash-flow planning.
Low-order seasonality (Fourier order 3, regularized)	Prophet's default yearly seasonality (10 Fourier terms) overfits 24 monthly points so badly it forecast *negative* revenue (backtest MAPE ~79%). A regularized low-order term brings MAPE to 1.36%.
Holt-Winters fallback	Prophet's compiled backend can fail to build on slim deploy hosts; the fallback keeps the demo alive with an identical output contract.

2.3 Layer 3 — small LLM (generative)

Choice	Why
Claude Haiku / GPT-4o-mini	Short, structured per-tenant generations at volume — a small fast model is the right cost/latency point; the default is `claude-haiku-5-5`.
Provider-agnostic + deterministic fallback	The engine resolves Anthropic → OpenAI → rule-based from the environment, so it runs for a reviewer with no API key.

---

3. Data pipeline

3.1 Why synthetic data

Cynbit's production data is not public, so the PoC generates a portfolio with the *same schema* TenantEase stores and realistic behavioural structure. The generator is causal, not random:

```
latent reliability (Beta 5,2)
      │  × seasonal pressure  − rent-burden penalty  + tenure bonus
      ▼
 per-cycle risk  ──►  P(miss) = 0.55·risk^2.2     (Bernoulli)
                 └──►  days late ~ Exp(0.9 + risk^2.1·44)
```

Both responses are **convex in risk**, which gives the classifier a learnable signal through noisy proxies — the same situation a production model faces. Calibration targets (real rent-roll benchmarks) are met: collection 94.5%, on-time 71.5%, missed cycles 5.9%.

3.2 Schema

- **tenants** — profile, unit, property type, rent, deposit, tenure, income.
- **payments** — one row per tenant per rent cycle: amount due/paid, days past due, paid flag, late flag. 520 tenants × up to 24 cycles = ~8,500 rows.
- **monthly_revenue** — aggregated demanded/collected, occupancy, late/missed.

3.3 Feature engineering and the temporal split

The single most important design decision. The ledger is split on the **time axis** so features never read the label window:

```
├◄──── observation window (21 cycles) ────►├◄── label window (3) ──►┤
        14 behavioural + profile features          will_default
```

- **14 features**: on-time rate, late rate, late count (6 cycles), missed count, avg/max/std days past due, 3-month delay trend, days since last payment, cycles observed, rent, rent-to-income, deposit months, tenure (+ booleans and one-hot property type → 22-column matrix).
- **Label**: a tenant is a default only with **2+ delinquent cycles** (missed or 15+ days late) in the label window — a *pattern*, not one unlucky month. This is both more realistic and a cleaner signal than a single-cycle trigger.

This split is enforced by a test (`test_no_temporal_leakage`) that asserts the observation and label windows are disjoint.

---

4. Prompt engineering (Layer 3)

Element	Approach
Role	System prompt casts the model as a property-management advisor for the Indian rental market, so tone and suggestions fit the domain.
Context injection	The tenant's risk score, payment behaviour, and profile are injected as **JSON**, not prose — structured facts the model reads directly.
Output contract	The model must return a single JSON object with exactly `{action, reason, urgency, suggested_message}`; `urgency` is constrained to four levels.
Validation & failure handling	The response is parsed (tolerating markdown fences), the schema is validated, and `urgency` is snapped to the nearest valid level. Any malformed output or API error **degrades to the deterministic generator** rather than surfacing a broken result.
Caching	Results are cached per tenant ID so dashboard re-renders don't re-bill the API.

5. Evaluation methodology

5.1 Risk model results

Metric	Value	Reading
Test AUC	**0.893**	Held-out 20%, never seen in training or threshold tuning
CV AUC	0.866 ± 0.080	5-fold stratified
Logistic baseline (test)	0.864	XGBoost lift +0.030
Oracle ceiling	0.918	Bayes-optimal (true latent trait); model sits **below** it → no leakage
Recall @ threshold 0.11	0.625	Threshold tuned on out-of-fold train predictions (max F1)

The **oracle AUC** is a deliberate rigor check: it is the AUC obtainable from the true hidden reliability trait. A model scoring *above* it would prove leakage; we sit below it, confirming the pipeline is honest.

5.2 Forecast evaluation

Backtest: hold out the last 3 observed months, forecast from the rest, report MAPE. Result: **1.36%** (Prophet). The confidence band is validated to bracket the point forecast; forecasts are asserted positive (guarding the overfit mode).

5.3 Test suite

34 tests across data, features, model, forecasting, and LLM layers — including the leakage guard and the no-API-key fallback guarantee. CI runs lint + tests on every push.

6. Limitations (stated plainly)

- **Synthetic data.** Results demonstrate capability, not production performance. Real behaviour will differ; a pilot on real TenantEase data is the next step.
- **Thin time series.** 24 monthly points is little for any seasonal model — hence the heavy regularization. Production would use daily collection data.
- **No fairness audit yet.** Features exclude protected attributes, but a production deployment must test for disparate impact (e.g. by property type or locality) before acting on scores.
- **Single-portfolio scope.** The PoC models one portfolio; multi-tenant SaaS would need per-landlord models or a pooled model with landlord features.
- **LLM cost at scale.** Per-tenant generation is cheap on a small model but adds up; the cache and a batch path mitigate this (§7).

7. Cost and scalability

Concern	Approach
Inference latency	Risk scores are precomputed and cached in Redis; the dashboard reads, never retrains.
Retraining cost	Monthly, offline, on a scheduled job — not in the request path.
LLM cost	Small model (Haiku/4o-mini), per-tenant caching, and a batch option for bulk nightly generation. Only high-risk tenants need fresh recommendations.
Horizontal scale	The FastAPI service is stateless (state in Postgres/Redis), so it scales behind a load balancer.
Graceful degradation	Every layer has a fallback (Holt-Winters for forecasting, rule-based for recommendations), so a dependency outage degrades quality, not availability.

8. Execution plan

Phase	Deliverable
1	Risk-scoring microservice on real TenantEase data (highest ROI)
2	Revenue forecasting with daily collection data
3	LLM recommendations wired into the existing reminder system
4	Fairness audit + monitoring (drift, calibration) before full rollout

The proof of concept in this repository implements phases 1–3 end to end on synthetic data; production hardening (phase 4, real-data retraining, monitoring) is the path to deployment.

Appendix — reproducing the results

```
python -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python scripts/generate_data.py     # synthetic portfolio
.venv/bin/python scripts/train.py             # risk model + metrics
.venv/bin/python scripts/forecast_revenue.py  # revenue forecast
.venv/bin/streamlit run app.py                # dashboard
.venv/bin/pytest                              # 34 tests
```